

COMPREHENSIVE PROJECT REPORT: DEEPPFAKE AUDIO DETECTION SYSTEM

1. Executive Summary

The rapid evolution of generative artificial intelligence has made it remarkably simple to clone human voices and synthesize highly convincing fake audio. This project presents a robust, end-to-end Machine Learning pipeline developed to counter this threat. By treating deepfake detection as an acoustic pattern recognition problem, we trained a deep 1D Convolutional Neural Network (CNN) to distinguish between genuine human speech and AI-generated variations.

The developed model achieves an **Overall Accuracy of 92.4%** and an **Equal Error Rate (EER) of 5.8%**, comfortably outperforming the required threshold parameters (≥ 80% accuracy and ≤ 12% EER). The final model is fully deployed to a cloud-based Streamlit application, providing an accessible, real-time interface for verification.

2. Problem Statement & Scope

Unregulated synthetic voice clones present high-level risks, including financial fraud (via voice phishing), political disinformation, and identity theft. The scope of this project is to build an automated software solution that accepts a single-channel audio recording, analyzes its underlying spectral properties, and provides a deterministic classification verdict (Genuine vs. Deepfake) alongside a statistical confidence score.

3. Dataset Profile & Preprocessing

3.1. Dataset Description

The model was trained, validated, and tested using **The Fake-or-Real Dataset (for-norm)**. This dataset contains thousands of pairs of human speech samples paired with their corresponding AI-synthesized deepfakes, generated across various modern text-to-speech (TTS) and voice conversion models.

3.2. Audio Standardization Pipeline

Raw audio formats are inherently heterogeneous. To guarantee structural uniformity prior to tensor conversion, the following mathematical and digital signal processing (DSP) constraints were enforced:

- **Sampling Rate Standard:** All audio streams are loaded and resampled uniformly to **16,000 Hz (16kHz)**. This captures up to an 8kHz Nyquist frequency, which preserves essential human vocal formants while filtering out unnecessary high-frequency noise.

- **Temporal Normalization:** Neural networks require static input tensors. Audio vectors are dynamically forced to a length of exactly **400 time frames** using the following conditional rule:

$$\text{Input Vector} = \begin{cases} \text{Truncate}(x, 400) & \text{if } \text{length}(x) > 400 \\ \text{PadZeros}(x, 400) & \text{if } \text{length}(x) < 400 \end{cases}$$

4. Feature Engineering: MFCC Extraction

Instead of feeding raw amplitude waveforms directly into the neural network, the system translates the time-domain signal into the frequency domain. We extract **Mel-Frequency Cepstral Coefficients (MFCCs)** to expose micro-anomalies introduced by AI vocoders.

[Raw Audio (.wav)] —> [Short-Time Fourier Transform] —> [Mel-Scale Filterbank Mapping] —> [Discrete Cosine Transform] —> [40 MFCC Coefficients]

- **Coefficients Extracted:** 40 distinct coefficients are computed per frame.
- **Justification:** While standard speech recognition models typically use 13 MFCCs to capture general linguistic content, deepfake detection requires a much higher resolution (40 coefficients) to capture high-frequency synthetic artifacts and unnatural spectral phase shifts.
- **Final Feature Shape:** Each audio track is resolved into an array of shape (400, 40).

5. Model Architecture & Training Strategy

5.1. Network Topology

A 1D Deep Convolutional Neural Network was selected over traditional Recurrent Neural Networks (RNNs) due to its superior computational speed on GPUs and its ability to extract invariant localized features across temporal frames.

Layer Type	Output Shape	Filter / Kernel / Rate	Functional Purpose
Input Layer	(None, 400, 40)	-	Receives standardized MFCC time-steps
Conv1D (Block 1)	(None, 400, 64)	64 Filters, Kernel=3, ReLU	Captures low-level spectral transitions

Layer Type	Output Shape	Filter / Kernel / Rate	Functional Purpose
Batch Normalization	(None, 400, 64)	-	Stabilizes internal covariate shift
MaxPooling1D	(None, 200, 64)	Pool Size = 2	Reduces spatial dimensions by 50%
Dropout	(None, 200, 64)	Rate = 0.3	Limits co-dependency between neurons
Conv1D (Block 2)	(None, 200, 128)	128 Filters, Kernel=3, ReLU	Extracts intermediate feature hierarchies
Batch Normalization	(None, 200, 128)	-	Normalizes activations across batch
MaxPooling1D	(None, 100, 128)	Pool Size = 2	Down-samples feature maps
Dropout	(None, 100, 128)	Rate = 0.3	Regularizes deeper layers
Conv1D (Block 3)	(None, 100, 256)	256 Filters, Kernel=3, ReLU	Assembles complex global acoustic maps
Batch Normalization	(None, 100, 256)	-	Prevents gradient explosion
MaxPooling1D	(None, 50, 256)	Pool Size = 2	Final feature reduction stage
Dropout	(None, 50, 256)	Rate = 0.3	Final convolutional dropout

Layer Type	Output Shape	Filter / Kernel / Rate	Functional Purpose
Flatten	(None, 12800)	-	Flattens 3D tensor to a 1D vector
Dense (Fully Conn.)	(None, 128)	ReLU	Learns highly non-linear decision bounds
Dropout	(None, 128)	Rate = 0.4	Mitigates overfitting before prediction
Output Layer	(None, 1)	Sigmoid Activation	Outputs binary probability $[0.0, 1.0]$

5.2. Hyperparameters & Optimization

- Loss Function:** Binary Cross-Entropy, mapping a target of 0 to Genuine and 1 to Deepfake.
- Optimizer:** Adam optimizer initialized with a learning rate of $\alpha = 0.001$.
- Early Stopping:** Configured to monitor validation loss with a patience value of 5 epochs, restoring the best historical weights upon termination to avoid over-optimization.

6. Experimental Results & Performance Evaluation

The model was subjected to evaluation using an independent hold-out validation dataset. It achieved outstanding results across all target validation metrics.

6.1. Metrics Summary Table

The network comfortably passed both primary and secondary design thresholds:

Metric Evaluation Criteria	Threshold Target	Achieved System Score	Status Verdict
Overall Model Accuracy	$\geq 80.0\%$	92.4%	Passed
Equal Error Rate (EER)	$\leq 12.0\%$	5.8%	Passed

Metric Evaluation Criteria	Threshold Target	Achieved System Score	Status Verdict
F1 Evaluation Metric	$\geq 80.0\%$	91.9%	Passed
Genuine Classification Accuracy	$\geq 75.0\%$	93.1%	Passed
Deepfake Classification Accuracy	$\geq 75.0\%$	91.7%	Passed

6.2. Confusion Matrix Analysis

The exact mathematical distribution of predictions indicates exceptional class balance, confirming that the network has not developed an unweighted bias toward either real or fake audio structures:

	Predicted: Genuine (0)	Predicted: Deepfake (1)
Actual: Genuine (\$0\$)	True Negatives (TN): 93.1%	False Positives (FP): 6.9%
Actual: Deepfake (\$1\$)	False Negatives (FN): 8.3%	True Positives (TP): 91.7%

6.3. Interpretation of Equal Error Rate (EER)

The achieved EER of **5.8%** is highly significant. In biometrics and audio verification, the EER represents the specific threshold setting where the probability of a False Acceptance Error equals the probability of a False Rejection Error. An EER below 6% indicates that the underlying MFCC features contain distinct, highly separated clusters for human and synthetic speech patterns.

7. Deployment & Interface Design

To translate this model into an accessible enterprise tool, the compiled weights file (deepfake_audio_model.keras) was integrated into a production-grade, lightweight web system.

[User Web Browser] —> [Uploads .wav File] —> [Streamlit Cloud Engine (Python 3.11)]
—> [Preprocesses MFCC & Runs Inference via TensorFlow-CPU] —> [Returns Metric
Dashboard & Waveform Plot]

7.1. Front-end Architecture

The user interface is powered by **Streamlit Community Cloud**, hooked directly to a public GitHub code repository.

- **File Uploader Widget:** Restricts incoming payloads exclusively to the .wav container format to ensure codec compatibility with librosa.
- **Visualization Engine:** Embeds a native matplotlib engine running librosa.display.waveshow to render an automated audio amplitude waveform graph dynamically upon upload.
- **State Management:** Utilizes the @st.cache_resource decorator to load the deep learning network once into memory upon server initialization, preventing the system from reloading the massive file on subsequent user file interactions.

7.2. Cloud Hosting Profile

To ensure the app remains accessible to stakeholders without manual compilation headaches, it is configured with a packages.txt file targeting libsndfile1 to run natively on headless Debian Linux servers, running a stable **Python 3.11 execution runtime**.

8. Conclusion & Future Enhancements

The development goals for this project have been fully realized. By isolating 40 distinct MFCC coefficients and routing them through a regularized, multi-layered 1D CNN, we constructed an engine capable of flagging synthetic speech with high fidelity.

Recommendations for Future Expansion:

1. **Acoustic Data Augmentation:** Introducing artificial room reverberations, background crowd noise, and network packet loss to training samples to increase model robustness against low-quality phone recordings.
2. **Transformer Integration:** Experimenting with specialized self-attention mechanics (e.g., Audio Spectrogram Transformers) to evaluate long-range temporal dependencies in longer files.